

Défis facultatifs

Ces défis sont **facultatifs et non notés**. Votre priorité est de terminer le **projet Fondation**, à rendre au plus tard à 16h30.

Dernière série ! Aucune notion nouvelle. Ces trois défis mobilisent **tout ce que vous avez appris** et sont un excellent entraînement pour l'examen écrit de 16h30.

Règles du jeu

- Vous codez **vous-mêmes, sans IA** : ni ChatGPT, ni Claude, ni autocomplétion Copilot.
- La documentation Python est autorisée et même encouragée.
- Un fichier par défi : `challenge5.py` , `challenge6.py` et `challenge7.py` .
- **Validation** : levez la main, exécutez votre code devant moi puis répondez à une question sur votre code.
- **Notation** : aucune, ces défis sont facultatifs. Faites-les valider quand même. C'est un bon entraînement pour l'examen.

5

Audit de configuration SSH

Fichiers · chaînes · dictionnaires · JSON

Audit

Lors d'un audit technique, on vérifie que la configuration des serveurs respecte la politique de sécurité de l'entreprise. Le serveur SSH de `srv-web` vous semble mal durci. À vous d'écrire l'outil qui le prouve.

Votre mission

1. Écrivez le texte `SSHD_CONFIG` ci-dessous dans un fichier `sshd_config`.
2. Écrivez la fonction `parse_config(path)`, qui relit le fichier et renvoie un dictionnaire `{paramètre: valeur}`. Les lignes vides et les commentaires (qui commencent par `#`) sont ignorés.
3. Écrivez la fonction `audit(config, policy)`, qui renvoie une liste de constats, un par règle de `POLICY`: un dictionnaire avec le paramètre, la valeur attendue, la valeur trouvée et un statut `"OK"`, `"NON CONFORME"` ou `"ABSENT"`. Quand la valeur attendue est un **entier**, c'est un **maximum**.
4. Écrivez la fonction `print_report(findings)`, qui affiche le rapport, puis enregistrez les constats dans `audit.json`.

Données à copier

```
SSHD_CONFIG = """# Configuration du serveur SSH de srv-web
Port 22
PermitRootLogin yes
PasswordAuthentication yes
PermitEmptyPasswords no

MaxAuthTries 6
X11Forwarding yes
# UsePAM yes
"""

POLICY = {
    "PermitRootLogin": "no",
    "PasswordAuthentication": "no",
    "PermitEmptyPasswords": "no",
    "MaxAuthTries": 3,          # un entier : valeur maximale autorisée
    "X11Forwarding": "no",
    "LogLevel": "VERBOSE",
}
```

Exemples attendus

Résultats attendus

```
config = parse_config("sshd_config")
print(config["MaxAuthTries"])      # 6

findings = audit(config, POLICY)
print_report(findings)
# [NON CONFORME] PermitRootLogin : yes (attendu : no)
# [NON CONFORME] PasswordAuthentication : yes (attendu : no)
# [OK] PermitEmptyPasswords : no
# [NON CONFORME] MaxAuthTries : 6 (attendu : au plus 3)
# [NON CONFORME] X11Forwarding : yes (attendu : no)
# [ABSENT] LogLevel (attendu : VERBOSE)
# Conformité : 1 / 6
```

Indices

- `line.split(maxsplit=1)` coupe la ligne en deux au premier espace seulement. On obtient le nom du paramètre puis sa valeur.
- `config.get(setting)` renvoie `None` si le paramètre est absent, au lieu de lever une `KeyError`.
- `isinstance(expected, int)` vous dit s'il faut comparer des nombres. Attention : `"10" <= "3"` est vrai entre chaînes !

Pour valider, montrez-moi...

- le rapport ci-dessus, qui donne exactement ce résultat ;
- le fichier `audit.json` ouvert dans VS Code.

Bonus

- Ne tenez pas compte de la casse des noms de paramètres, comme le fait le vrai serveur SSH.
- Sur une machine Linux, auditez le vrai fichier `/etc/ssh/sshd_config`, en lecture seule.

6

Chiffre de César

Boucles · % · fonctions · chaînes

Cryptologie

Jules César chiffrait ses messages en décalant chaque lettre de quelques positions dans l'alphabet. Vous avez intercepté un message chiffré de cette façon, sans connaître le décalage. Montrez qu'un chiffrement avec seulement 26 clés possibles ne protège rien.

Votre mission

1. Écrivez la fonction `encrypt(text, shift)`, qui décale chaque lettre de `shift` positions. Les majuscules restent en majuscules et les autres caractères (espaces, apostrophes...) ne changent pas. Après `z`, on revient à `a`.
2. Écrivez la fonction `decrypt(text, shift)` ... en une seule ligne, grâce à `encrypt`.
3. Écrivez la fonction `crack(ciphertext)`, qui teste **les 26 décalages** et renvoie le décalage et le texte les plus probables : ceux qui contiennent le plus de mots de `COMMON_WORDS`.

Données à copier

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
COMMON_WORDS = [
    "le", "la", "les", "de", "des", "du", "et",
    "est", "un", "une", "dans", "pour", "mot", "passe",
]
SECRET = "Wp xze op alddp of dpcgpfc op dlfgprlcop pde olyd wp nzqqcp"
```

Exemples attendus

Résultats attendus

```
print(encrypt("Attaque a l'aube", 3))    # Dwwdtxh d o'dxeh
print(decrypt("Dwwdtxh d o'dxeh", 3))    # Attaque a l'aube
print(encrypt("xyz", 3))                  # abc

print(crack(SECRET))
# (décalage, message) : à vous de les découvrir !
```

Indices

- `ALPHABET.index("c")` donne la position d'une lettre (ici `2`) ; `ALPHABET[4]` fait l'inverse.
- L'opérateur `%` fait revenir au début de l'alphabet : `(25 + 3) % 26` vaut `2`.
- Pour une majuscule, travaillez sur `char.lower()` puis remettez en majuscule avec `upper()`.

Pour valider, montrez-moi...

- les trois premiers exemples, qui donnent exactement ces résultats ;
- le message secret, retrouvé **automatiquement** par `crack` .

Bonus

- **Analyse de fréquence** : en français, la lettre la plus fréquente est le « e ». Retrouvez le décalage sans liste de mots.
- Implémentez le **chiffre de Vigenère**, qui utilise un mot-clé au lieu d'un seul décalage. Est-il beaucoup plus solide ?



Tableau de bord d'alertes

Classes • héritage • JSON • dictionnaires

SOC

Le SOC reçoit chaque matin un rapport d'alertes au format JSON. Votre mission : le recharger, recréer les objets du défi 4 puis afficher un **tableau de bord** pour l'équipe d'astreinte.

Votre mission

1. Placez votre `challenge4.py` du jour 2 dans le même dossier. Écrivez le rapport `REPORT` ci-dessous dans `report.json`.
2. Écrivez la fonction `alert_from_dict(data)`, qui recrée le **bon type d'objet** selon la clé `type` : `BruteForceAlert`, `CompromiseAlert` ou `Alert`.
3. Écrivez la fonction `load_alerts(path)`, qui relit `report.json` et renvoie la liste des objets.
4. Écrivez la fonction `dashboard(alerts)`, qui affiche le nombre d'alertes par sévérité, l'IP la plus active et les comptes compromis.

Données à copier

```
REPORT = [  
    {"type": "brute_force", "ip": "203.0.113.45", "failures": 7},  
    {"type": "brute_force", "ip": "198.51.100.23", "failures": 5},  
    {"type": "compromise", "ip": "203.0.113.45", "user": "Chris"},  
    {"type": "info", "ip": "192.0.2.99"},  
    {"type": "brute_force", "ip": "203.0.113.45", "failures": 12},  
    {"type": "compromise", "ip": "198.51.100.23", "user": "backup"},  
    {"type": "info", "ip": "192.0.2.25"},  
]
```

Exemples attendus

Résultats attendus

```
alerts = load_alerts("report.json")
print(type(alerts[2]).__name__)    # CompromiseAlert

dashboard(alerts)
# Alertes : 7
#   CRITICAL : 2
#   HIGH : 3
#   INFO : 2
# IP la plus active : 203.0.113.45 (3 alertes)
# Comptes compromis : Chris, backup
```

Indices

- `from challenge4 import Alert, BruteForceAlert, CompromiseAlert` fonctionne si les deux fichiers sont dans le même dossier.
- Comptez avec des dictionnaires : un pour les sévérités et un pour les IP. Parcourez ensuite celui des IP pour trouver la plus fréquente.
- `isinstance(alert, CompromiseAlert)` repère les comptes compromis.

Pour valider, montrez-moi...

- les deux exemples ci-dessus, qui donnent exactement ces résultats ;
- votre fonction `alert_from_dict` . Expliquez-moi aussi pourquoi recréer des objets plutôt que garder des dictionnaires.

Bonus

- Affichez les alertes de la plus grave à la moins grave, grâce à leur méthode `describe()` .
- Modifiez le `to_dict()` du défi 4 pour qu'il produise exactement ce format. L'aller-retour entre les défis 4 et 7 est alors complet.